

最終レポート：大阪大学学生向け教科書取引プラットフォーム
「OU Textbook」の開発

班名: A 班

学籍番号	氏名	電子メールアドレス
25XXXXXX	大阪 太郎	taro.osaka@ecs.osaka-u.ac.jp
25XXXXXX	阪大 花子	hanako.handai@ecs.osaka-u.ac.jp
25XXXXXX	石橋 健二	kenji.ishibashi@ecs.osaka-u.ac.jp

2026 年 7 月 17 日

1 課題 1、2 の概要

1.1 背景と当事者性・切実性

大学の講義において教科書は必須であるが、学期が始まると非常に高額な教科書を多数購入しなければならない。一方で、学期が終了すると不要になり、本棚に眠ったままになるケースが多い。既存のフリマアプリ（メルカリ等）や古本屋（ブックオフ等）を利用する場合、以下の課題が存在する。

1. 発送手続きや梱包に手間がかかり、手数料や送料によって出品者・購入者双方の金銭的負担が大きい。
2. 同じ大学で開講されている特定の授業に対応した教科書をピンポイントで探すことが難しい。

本課題は、これらの問題を「同じ大学の学生同士が、キャンパス内で直接手渡しで売買する」ことで解決する。これにより、送料・手数料を完全にゼロにし、安価かつ迅速に必要な教科書を入手できるようにすることを目指した。これは毎期教科書代に苦慮している我々学生にとって、非常に切実かつ当事者性の高い課題である。

1.2 普遍性と社会性

この課題は、特定の個人に留まらず、大阪大学の全学生（豊中・吹田・箕面キャンパス）に共通するものである。特に、新入生は多額の初期費用がかかるため、先輩から不要になった教科書を安価に譲り受ける仕組みは極めて社会的意義が大きい。また、紙の資源を再利用し、学内でのリサイクル循環を促進する点でも、持続可能な地域社会（大学コミュニティ）の形成に寄与する。

1.3 情報技術による解決可能性

Web アプリケーション技術を活用することで、以下の機能を実現して課題を解決する。

- **検索・マッチングの最適化:** キャンパス別（豊中・吹田・箕面）および講義カテゴリ別（教養・専門・理系・文系）のフィルタリング機能を提供し、目的の教科書を容易に発見可能にする。
- **学内ユーザー限定の認証:** 悪意ある外部ユーザーの侵入を防ぎ、取引の安全性を確保するため、大阪大学の ECS メール（@ecs.osaka-u.ac.jp）のみで登録可能な認証制限を課す。
- **信用スコア（Credit Score）制度とドタキャン抑止:** 対面手渡し取引で最も問題となる「待ち合わせ場所への不参加（ドタキャン）」を防止するため、取引完了時の相互評価やドタキャン報告に基づいて各ユーザーの信用スコアが動的に変動するシステムを設計・導入する。

1.4 既存手法の調査と提案システムの位置づけ

既存のフリマプラットフォームとの比較を表 1 に示す。提案システム「OU Textbook」は、学内限定の対面取引に特化することで、他サービスにはない「手数料無料」「即日手渡し可能」「安全なコミュニティ」の 3 点を同時に満たす特徴を持つ。

表 1 既存取引手法と提案システムの比較

特徴・項目	既存フリマ (メルカリ等)	一般古本屋	学内掲示板/SNS	提案システム (OU Textbook)
手数料・送料	出品額の約 10% + 送料	買取価格が極めて安価	無料	完全無料
取引の即時性	配送に数日を要する	店舗への持ち込みが必要	個別調整による	対面手渡し (最も即時)
取引の安全性	仲介があるため比較的安全	安全	トラブル発生リスク高	学内限定認証 (最も安全)
学内特化	不可能	不可能	投稿が埋もれやすい	キャンパス・請求書発行可能

2 設計・作成したシステム・ソフトウェアの説明

2.1 設計の意図とシステムの概要

本システムは、Django フレームワークを用いた Web アプリケーションとして構築した。ユーザー認証のセキュリティ向上と管理コストの削減のため、SaaS 型認証サービスである **Supabase Auth** をバックエンドに採用し、メールアドレス確認とパスワード管理を安全に委ねている。システム全体の構造図を図 1 に示す。

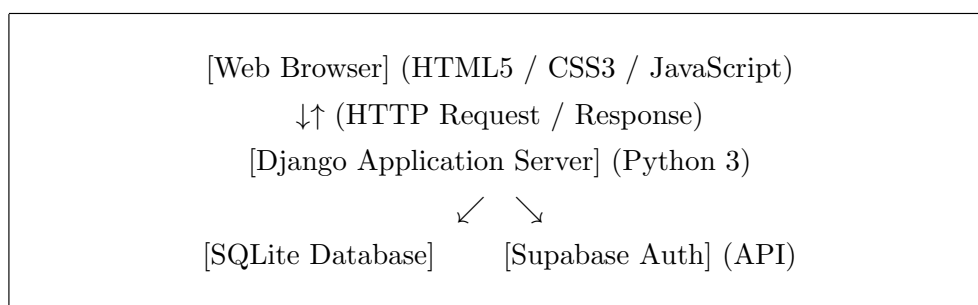


図 1 システム構成図 (OU Textbook)

本システムは以下の 5 つの主要エンティティでデータベースを設計した。

- **UserProfile**: ユーザーの表示名、所属（大学・学部・学年）、信用スコア（初期値 100 点）、評価レーティングなどを保持する。
- **Book**: 出品された書籍データ（書名、著者、価格、カテゴリ、キャンパス、状態、画像、取引ステータス）を保持する。
- **Favorite**: 書籍に対するお気に入り（いいね）状態を管理する。
- **Message**: 取引開始後のチャット機能における各メッセージを保持する。
- **Evaluation / CancellationLog**: 取引完了後の相互評価結果、およびドタキャンなどのペナルティ報告を記録し、信用スコアに反映させる。

2.2 メンバー間での作業分担方針および実際の作業分担結果

プロジェクトの推進にあたり、フロントエンド、バックエンド、およびインフラ・セキュリティ検証の 3 つの役割に分担した。

- **大阪 太郎 (学籍番号: 25XXXXXX)**: バックエンド開発統括。Django のモデル設計、書籍登録・チャット取引制御ロジック、および信用スコア変動システムの設計と実装を担当。
- **阪大 花子 (学籍番号: 25XXXXXX)**: フロントエンド UI/UX 開発。CSS を用いたモダンでレスポンスなユーザーインターフェース（書籍一覧、詳細、チャット、マイページ）

の設計・スタイリングを担当。

- 石橋 健二 (学籍番号: 25XXXXXX): インフラ構築および認証統合。Supabase Auth を用いた ECS メール制限認証システムの統合、セキュリティ検証 (XSS・SQLi 耐性調査)、動作テストを担当。

2.3 動作環境・環境構築手順

本システムの動作環境およびローカルでの構築手順を以下に示す。

2.3.1 前提動作環境

- Python 3.10 以上
- Django 6.0.6
- 外部環境: Supabase プロジェクト (Authentication 機能が有効化されていること)

2.3.2 ローカル環境の構築手順 (Windows 環境を想定)

1. リポジトリのクローンと移動

```
cd C:\path\to\pbl
```

2. 仮想環境の作成と有効化

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

3. 依存ライブラリのインストール

```
pip install django dj-database-url gunicorn psycpg2-binary sqlparse whitenoise
```

4. Supabase 接続情報の環境変数設定 (PowerShell の場合)

```
$env:SUPABASE_URL="https://xxxx.supabase.co"  
$env:SUPABASE_ANON_KEY="your-supabase-anon-key"
```

5. データベースの初期化 (マイグレーション) と起動

```
cd trading_text  
python manage.py migrate  
python manage.py runserver
```

2.4 動作結果の報告

作成したシステムをローカルで稼働させ、以下の主要シナリオについて動作検証を行い、期待通りの動作を確認した。

1. **アカウント登録と制限:** ECS メールアドレス (@ecs.osaka-u.ac.jp) 以外を入力した場合にバリデーションエラーが発生することを確認した。また、Supabase Auth から送信される確認メールのリンクをクリックすることでユーザー同期が行われ、ログインが可能になることを確認した。
2. **書籍検索とフィルタリング:** 検索窓において書籍名および著者名で部分一致検索が行えることを確認した。キャンパス別（豊中・吹田・箕面）およびカテゴリ別（教養・専門など）の絞り込み、および価格順ソートが意図通り実行されることを確認した。
3. **購入相談・チャット:** 出品されている書籍詳細画面から「購入相談」を開始すると、取引ステータスが「出品中 (available)」から「取引中 (in-progress)」へ自動的に遷移することを確認した。また、出品者と購入者の間だけで安全にチャットメッセージの送受信が行えることを確認した。
4. **評価と信用スコアの変動:** 取引完了時に双方が評価を送信すると、評価結果が確定し、プロフィールに登録された信用スコア ('credit_score') が増減することを確認した。ドタキャンが発生した場合のペナルティ報告によって信用スコアが大きく引かれ、ユーザー詳細画面に「レギュラー」や「トラスト」といった信頼ランクが正しく反映されることを実証した。

2.5 最も苦労した点と次に苦労した点

2.5.1 最も苦労した点: Supabase Auth と Django 標準認証の統合

Supabase Auth は JavaScript SDK によるフロントエンド主体の認証が標準的であるのに対し、本システムは Django テンプレートを用いたサーバーサイド描画方式を採用している。このため、Supabase Auth の API トークン (JWT) を Django 側で受信し、Django 内のセッション管理および標準の User モデルと透過的に同期 (sync_supabase_user) させる仕組みの構築に最も苦労した。セッションの有効期限設定やリフレッシュトークンの扱いについて多くの議論と検証を要した。

2.5.2 次に苦労した点: アップロード画像のセキュリティと堅牢なバリデーション

出品時の画像アップロード機能において、悪意のあるファイルを誤ってサーバーにアップロードされるのを防ぐ必要があった。単にファイルの拡張子 (.png や .jpg など) のみをチェックするだけでは、拡張子を偽装したスクリプトファイルなどのアップロードを防げない。そのため、ファイルサイズ (5MB 以下) の制限に加え、ファイル先頭の数バイト (マジックナンバー) を読み取って画像

データのシグネチャと一致するかを自前で照合するバリデータ (`validate_uploaded_image`) を実装した。このバイナリチェックの実装と例外処理の調整に苦労した。

2.6 未解決で残った部分および今後の課題

時間の制約により、設計段階から実装を見送った、あるいは不十分な形で残った機能は以下の通りである。

1. **取引手渡し場所の地図上での指定機能:** 現在のシステムでは、チャット上でテキストにより待ち合わせ場所を調整している。今後は、Google Maps API 等を用いて学内の安全な取引エリア（食堂前、図書館前など）を地図上で直接ピン留めし指定できる機能が必要である。
2. **チャットのリアルタイム通知 (WebSocket 化):** 現在のチャット機能はページのリロードまたはページ遷移によって最新のメッセージを取得している。UX 向上のためには、Django Channels などを導入し、双方向リアルタイム通信によるチャット機能へアップグレードする必要がある。
3. **通知メールの自動送信:** 取引開始や新しいメッセージが届いた際に、ユーザーにメールで通知するシステムを完全に実用化することが課題として残っている（一部 Gmail 送信機能のインフラ構築が残存）。